

Introduction to RISC-V Architecture using RIPES Simulator

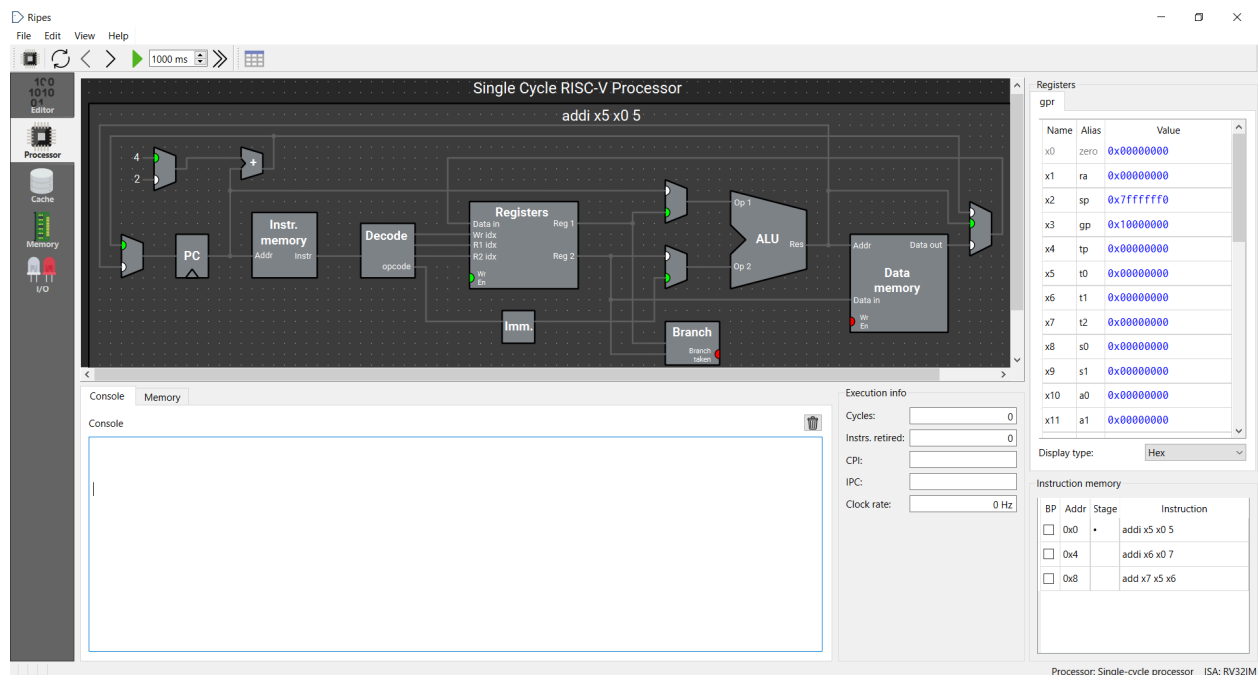
Learning Objectives

- Understand how **RISC-V instructions** are executed.
- Visualize **data flow and control signals** inside the processor.
- Compare **CISC and RISC architectures** practically.

Introduction to RIPES

After working with **EMU8086** to understand the fundamentals of **CISC (Complex Instruction Set Computer)** architecture, we now move towards exploring the **RISC (Reduced Instruction Set Computer)** architecture through a visual simulation tool called **RIPES**.

RIPES is an interactive processor simulator designed for **educational purposes**, helping students visualize how a **RISC-V processor** executes instructions. It allows you to see how data moves through different parts of the CPU—such as the **register file, memory, ALU, and control unit**—step by step.



Unlike EMU8086, which runs assembly code for an Intel-based CISC processor, RIPES focuses on the **RISC-V architecture**, which is simpler, modular, and widely used in modern processors and embedded systems. By working in RIPES, you will gain a deeper understanding of how each instruction affects the processor state and how pipelining improves performance.

Why We Use RIPES

RIPES bridges the gap between **assembly programming** and **computer architecture concepts**. It not only allows you to write and run assembly programs but also helps you see what happens *inside* the CPU during each instruction cycle — something that is abstracted away in EMU8086.

Key Features of RIPES

- Supports **RISC-V** instruction set simulation (RV32I and others)
- Real-time visualization of the **pipeline stages** (Fetch, Decode, Execute, Memory, Writeback)
- Displays **register contents, memory, and control signals**
- Step-by-step or continuous execution modes
- Graphical interface for easy learning and experimentation

Register Details

Name	Alias	Purpose / Convention
x0	zero	Always 0 (hardwired constant zero)
x1	ra	Return address
x2	sp	Stack pointer
x5–x7	t0–t2	Temporaries
x10–x17	a0–a7	Function arguments / return values
x28–x31	t3–t6	More temporaries

Hardware and Conventional Register Naming in RISC-V

Aspect	x Registers	Conventional Names (t, a, s, etc.)
Meaning	Hardware register numbers used by the CPU	Human-readable aliases indicating purpose
Example	x6	t1
Relationship	One-to-one (e.g., t1 = x6)	Same physical register, named by usage convention
Purpose	Machine-level identification	Code clarity and readability
Use Case	Used in low-level hardware docs or encodings	Used in assembly programming for easier understanding

First Program

```
addi t0, x0, 5 # Load immediate value 5 into t0
addi t1, x0, 7 # Load immediate value 7 into t1
add t2, t0, t1 # t2 = t0 + t1
```

Output

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000000
x2	sp	0x7fffffff0
x3	gp	0x10000000
x4	tp	0x00000000
x5	t0	0x00000005
x6	t1	0x00000007
x7	t2	0x0000000c

Instruction Design Philosophy: RISC vs. CISC Approaches

CISC Instruction: In CISC architectures (e.g., 8086), instructions like `MOV AL, 5` directly load constants into registers without needing an intermediate or source operand. It has following features.

- The instruction formats are variable in length and flexible in operand specification (register, memory, or immediate).
- This allows a single, complex instruction to perform multiple low-level operations, reducing the total number of instructions required to accomplish a task.

However, such flexibility makes decoding and control hardware more complex and slower.

RISC instruction: In RISC architectures (e.g., RISC-V), the same operation is expressed as `ADDI t0, x0, 5`. Here, `x0` acts as a hardwired zero register, ensuring all arithmetic and immediate operations follow a fixed 3-operand format and has following properties

- This design keeps instruction formats simple, uniform, and easily decodable.
- RISC processors typically execute more instructions for the same task but benefit from faster clock speeds and efficient pipelining.

Philosophical contrast:

- CISC: Fewer, more capable instructions → increased hardware complexity.
- RISC: More, simpler instructions → reduced hardware complexity and improved execution speed.

Comparison of Operation per Instruction Cycle (RISC vs CISC)

Feature	RISC (Fixed Operation)	CISC (Variable Operation)
Instruction Complexity	Simple, performs one operation	Complex, may perform multiple operations
Execution Time	Usually one clock cycle per instruction	Varies depending on instruction type
Control Logic	Simple and fast	Complex and slower
Pipeline Efficiency	High (easy to pipeline)	Low (difficult to pipeline)
Example	<code>ADD R1, R2, R3</code> → adds two registers	<code>MOV [BX+SI], AX</code> → address + memory + move

RISC-V Instruction Syntax and Assembler Auto-Fill Behavior

- RIPES assembler auto-fills missing operands for simplicity.
- If a source register is not specified, it assumes **x0 (zero register)** by default.
- This allows shortened instructions to execute correctly inside RIPES.
- However, such shorthand is **non-standard** and may fail in official RISC-V assemblers.
- Standard RISC-V format always uses **three operands** per instruction.
 - `addi destination, source, immediate`
 - `add destination, source1, source2`
- Writing complete syntax ensures **clarity, correctness, and portability** across environments.

Involving subtraction

```
addi t0, x0, 5 # Load immediate value 5 into t0
addi t1, x0, 7 # Load immediate value 7 into t1
add t2, t0, t1 # t2 = t0 + t1
addi t3, x0, 2 # Load another value 2 into t3
sub t4, t2, t3 # t4 = t2 - t3 (12 - 2 = 10)
```

Explanation of the Code

- **addi t0, x0, 5** → Loads the immediate value **5** into register t0 ($t0 = 0 + 5$).
- **addi t1, x0, 7** → Loads the immediate value **7** into register t1 ($t1 = 0 + 7$).
- **add t2, t0, t1** → Adds the values in t0 and t1; result **12** is stored in t2 ($t2 = 5 + 7$).
- **addi t3, x0, 2** → Loads the immediate value **2** into register t3.
- **sub t4, t2, t3** → Subtracts t3 from t2; result **10** is stored in t4 ($t4 = 12 - 2$).

Conceptual Notes

- Demonstrates **immediate loading** and **register-based arithmetic**.
- All operations occur **within CPU registers** — no memory access.
- Illustrates RISC principle: each instruction performs a **single, simple task**.

- Uses x0 (zero register) to load constants since RISC-V has no direct MOV instruction.

Involving multiplication

```
addi t0, x0, 8    # Load immediate value 8 into t0
addi t1, x0, 2    # Load immediate value 2 into t1
mul  t2, t0, t1   # t2 = t0 * t1 (8 * 2 = 16)
addi t3, x0, 4    # Load another value 4 into t3
div  t4, t2, t3   # t4 = t2 / t3 (16 / 4 = 4)
addi t5, t4, 5    # Add 5 to result -> t5 = 4 + 5 = 9
```

Branch Example

```
addi t0, x0, 5    # Load 5 into t0
addi t1, x0, 7    # Load 7 into t1
blt  t0, t1, LESS # If t0 < t1, jump to LESS
addi t2, x0, 0    # False case: t2 = 0
jal  x0, END_LABEL # Unconditional jump to END_LABEL

LESS:
addi t2, x0, 1    # True case: t2 = 1

END_LABEL:
addi t3, x0, 9    # Dummy instruction to mark program end
```

Instructions details

- **blt** — *Branch if Less Than*: Jumps to the given label if the first register's value is **less than** the second's.
- **jal** — *Jump and Link*: Jumps to the given label and **links** (stores) the return address in the specified register (ignored if x0).

Explanation

- **Load values**: $t0 \leftarrow 5$ and $t1 \leftarrow 7$ using addi with base register x0.
- **Compare**: blt t0, t1, LESS checks if $t0 < t1$; if true, jump to label LESS.
- **False path**: If not less, set $t2 = 0$.

- **Jump over true block:** jal x0, END_LABEL skips the LESS block when the condition is false.
- **True path:** At LESS:, assign t2 = 1 to mark condition true.
- **End label:** END_LABEL: marks program continuation; addi t3, x0, 9 is a dummy operation showing normal flow resumes.

Exercise:

Task 1: Perform all mathematical operations in a single RISC-V assembly program.

Task 2: Explore gate operations (AND, XOR etc) and use in RISC-V assembly.

Task 3: Use other comparison (branch jump) instructions in an example in RISC-V assembly.